

GAMMA

Gobierno Automatizado del Maestro de Materiales

Manual de instalación

Puesta en marcha, actualización y operación

Unidad de Energía · 12 de agosto de 2026

Dirigido a quien despliega y mantiene la plataforma. El uso diario se documenta en el manual de usuario, y la administración en el manual técnico.

Índice

1. Requisitos	2
2. Servicios	3
2.1. El frontend como punto de entrada	3
2.2. Los dos modos del frontend	3
3. Variables de entorno	4
3.1. Los dos umbrales	4
4. Primera instalación	5
4.1. Obtener el código y configurar	5
4.2. Levantar los servicios	5
4.3. Aplicar las migraciones	5
4.4. Crear el primer administrador	5
4.5. Cargar los datos iniciales	5
5. Actualizar una instalación existente	6
5.1. Procedimiento	6
5.2. Por qué cada paso	6
6. Problemas frecuentes	7
7. Respaldos	9

1 Requisitos

- **Docker** con el complemento Compose. Es el único requisito obligatorio: todo lo demás corre dentro de contenedores.
- **Git**, para obtener y actualizar el código.
- Una **clave de acceso al servicio de modelo de lenguaje**. Es el único componente que no es de código abierto y el único que tiene costo, bajo un esquema de pago por uso.
- Alrededor de **4 GB de memoria disponible**. El modelo de clasificación se carga completo en memoria, y durante un reentrenamiento conviven dos en paralelo.

2 Servicios

Todo se orquesta con Docker Compose sobre una red interna.

Servicio	Puerto	Qué es
db	5436	PostgreSQL 16 con la extensión pg_trgm, que resuelve la búsqueda por similitud
api	8001	FastAPI sobre Python 3.12. Incluye el modelo de clasificación y la conexión al modelo de lenguaje
lab	8888	Jupyter Lab con los cuadernos de análisis
frontend	8089	La aplicación web. Sirve además de proxy hacia los demás servicios

2.1 El frontend como punto de entrada

En producción, el frontend no solo sirve la aplicación: unifica el acceso a todo bajo un mismo dominio.

Ruta	Destino
/	La aplicación web
/api/*	El API
/swagger-ui	La documentación interactiva del API
/lab/*	Jupyter Lab

2.2 Los dos modos del frontend

El comportamiento lo decide la variable FRONTEND_MODE:

- **development** — servidor de desarrollo con recarga automática. Los cambios en el código se reflejan sin reconstruir nada.
- **production** — compila la aplicación y sirve archivos estáticos. Más rápido, pero **cada cambio exige reconstruir la imagen**.

Cuál usar

En modo desarrollo una actualización se aplica con solo recrear el contenedor, lo que simplifica bastante el mantenimiento. Si opta por modo producción, recuerde que tras cada actualización hay que reconstruir: `docker compose up -d --build frontend`.

3 Variables de entorno

Se definen en un archivo `.env` en la raíz del proyecto. Parta de `.env.example`.

Variable	Para qué sirve
DB_USER, DB_PASSWORD	Credenciales de la base de datos
DB_NAME, DB_HOST, DB_PORT	Ubicación de la base
JWT_SECRET	Clave con la que se firman las sesiones. Debe ser un valor largo y aleatorio
JWT_EXPIRY_HOURS	Duración de la sesión
LLM_API_KEY	Clave del servicio de modelo de lenguaje
LLM_MODEL	Modelo de lenguaje a utilizar
JUPYTER_TOKEN	Token de acceso al laboratorio
CONFIANZA_UMBRAL	A partir de qué confianza una sugerencia se da por resuelta
DUPLICADO_UMBRAL	A partir de qué similitud se alerta de un posible duplicado
FRONTEND_MODE	development o production

No versione el archivo `.env`

Contiene credenciales y claves. Está excluido del repositorio a propósito. Cada entorno mantiene el suyo.

3.1 Los dos umbrales

Merecen atención porque cambian el comportamiento del asistente:

- `DUPLICADO_UMBRAL` está deliberadamente bajo. Bajarlo aún más genera más alertas irrelevantes; subirlo deja pasar duplicados reales, que son mucho más costosos.
- `CONFIANZA_UMBRAL` decide cuándo el sistema da por buena su propia sugerencia. Subirlo aumenta la exactitud pero deriva más casos a revisión manual.

4 Primera instalación

4.1 Obtener el código y configurar

```
git clone <repositorio> gamma
cd gamma
cp .env.example .env
```

Complete el .env con las credenciales y claves.

4.2 Levantar los servicios

```
docker compose up -d --build
```

La primera vez tarda varios minutos: descarga las imágenes base e instala dependencias.

4.3 Aplicar las migraciones

```
docker compose exec api python migrate.py --status
docker compose exec api python migrate.py
```

El primer comando lista qué hay aplicado y qué falta; el segundo aplica lo pendiente. Es idempotente: repetirlo no causa daño.

Los scripts de inicialización solo corren una vez

Los archivos de db/init/ se ejecutan únicamente cuando el volumen de datos está vacío. En una base ya creada no vuelven a correr: para eso están las migraciones.

4.4 Crear el primer administrador

Registre una cuenta desde la aplicación y promuévala. **Este paso es obligatorio:** sin él nadie puede entrar a las secciones de administración.

```
docker compose exec db psql -U <usuario> -d <base> \
-c "UPDATE public.users SET admin=true WHERE email='...';"
```

A partir de ahí, los demás roles se gestionan desde la sección Usuarios.

4.5 Cargar los datos iniciales

Desde la sección **Datos**, pestaña Importar, cargue en este orden: UNSPSC, luego el catálogo de clases y al final los archivos del maestro. El orden importa; el manual técnico explica por qué.

5 Actualizar una instalación existente

5.1 Procedimiento

```
cd /ruta/del/proyecto
git status          # revisar que no haya cambios locales sin guardar
git pull
docker compose exec api python migrate.py
docker compose up -d --force-recreate --no-deps frontend
```

5.2 Por qué cada paso

- **Las migraciones** deben correr siempre. Una pantalla que consulta una tabla que todavía no existe falla sin explicación clara.
- **Recrear el frontend** es necesario porque algunos archivos se montan individualmente en el contenedor, y al reemplazarlos el contenedor pierde la referencia. Se detalla más adelante.
- **El API no requiere nada:** su código se monta como directorio y el servidor recarga solo. Solo hay que reconstruirlo si cambiaron sus dependencias.

Reconstruir si cambian las dependencias

Si la actualización modificó las dependencias del API o de la aplicación web, sí hay que reconstruir la imagen correspondiente con `docker compose up -d --build`. Revise el registro de cambios ante la duda.

6 Problemas frecuentes

Tras actualizar, la aplicación muestra un error de archivo no encontrado

Es el problema más común y no significa que falte nada. Algunos archivos se montan individualmente en el contenedor, y Docker los sigue por su identificador interno, no por su nombre. Cuando la actualización reemplaza uno de esos archivos, el contenedor queda apuntando a algo que ya no existe.

```
docker compose up -d --force-recreate --no-deps frontend
```

Reiniciar no basta

`docker compose restart` reutiliza el mismo contenedor y conserva el montaje defectuoso. Hay que recrearlo.

El pull falla diciendo que las ramas divergieron

Ocurre si el historial del repositorio se reescribió. En un servidor, que solo consume código y no produce cambios, la salida es adoptar el remoto tal cual:

```
git fetch origin
git reset --hard origin/main
```

Revise antes de descartar

`git reset --hard` descarta cualquier cambio local en archivos versionados. Ejecute `git status` primero. Los archivos excluidos del repositorio, como `.env` y los artefactos del modelo, no se ven afectados.

El pull falla por archivos sin versionar que serían sobrescritos

Sucede cuando un archivo que antes estaba excluido pasa a formar parte del repositorio. Aparta los locales y vuelve a intentar:

```
mkdir -p ~/respaldo && mv <archivos> ~/respaldo/
git pull
```

Nadie puede entrar a las secciones de administración

Ninguna cuenta tiene el rol. Promueva una desde la base de datos, como se describe en la primera instalación.

Una pantalla falla al consultar datos

Casi siempre son migraciones sin aplicar. Verifique con `migrate.py --status`.

El asistente no responde

Revise que `LLM_API_KEY` esté configurada y que el servicio externo esté disponible. Los errores de comunicación quedan registrados y se pueden consultar desde la sección Datos.

7 Respaldos

Tres cosas merecen respaldo, y solo una está en el repositorio:

Qué	Por qué
La base de datos	Contiene el catálogo, las solicitudes, las conversaciones y las cuentas. Es lo único verdaderamente irrecuperable
El archivo .env	Credenciales y claves. No está versionado
Los artefactos del modelo	No viajan por el repositorio. Se pueden regenerar reentrenando, pero eso toma tiempo y no reproduce exactamente el modelo anterior

El código no necesita respaldo: vive en el repositorio.

Los artefactos son propios de cada entorno

Como no se versionan, cada instalación mantiene los suyos. Un entorno recién desplegado arranca con el modelo inicial y su propio historial de versiones.

Antes de recrear la base

`docker compose down -v` borra los volúmenes, y con ellos toda la información. Úselo solo cuando quiera partir de cero, y nunca sin respaldo previo.